LAPORAN PRAKTIKUM TEKNOLOGI WEB SERVICE
PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA

AUTENTIKASI DAN OTORISASI JSON WEB TOKEN (JWT) SERTA IMPLEMENTASI KEAMANAN DASAR RESTFUL API

Studi Kasus Pengembangan REST API Mahasiswa dengan Role-Based Access Control, Password Hashing Bcrypt, Rate Limiting, dan Audit Logging

Disusun untuk memenuhi tugas praktikum mata kuliah Teknologi Web Service (Pertemuan 7)

Nama Mahasiswa	:	**LAELA RUSTIANI**
NPM	:	**23670004**
Program Studi	:	Informatika
Tahun Akademik	:	2026 / 2027 (Semester Gasal)

SEMARANG
2026

KATA PENGANTAR

Puji dan syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa atas berkat, rahmat, dan karunia-Nya sehingga penyusunan Laporan Praktikum Pertemuan 7 Mata Kuliah Teknologi Web Service dengan judul **"Autentikasi dan Otorisasi JSON Web Token (JWT) serta Implementasi Keamanan Dasar RESTful API"** dapat diselesaikan dengan baik dan tepat waktu.

Laporan ini menyajikan hasil implementasi sistem proteksi web service berbasis REST API yang dibangun menggunakan bahasa pemrograman PHP native dan basis data MySQL. Fokus utama kegiatan praktikum dan tugas pengembangan ini meliputi pembuktian identitas menggunakan standar token tanpa status (*stateless token*) JWT RFC 7519, penerapan otorisasi berbasis peran (*Role-Based Access Control / RBAC*), perlindungan kata sandi menggunakan fungsi hash satu arah Bcrypt, penangkalan serangan injeksi melalui PDO Prepared Statements, pembatasan laju permintaan (*rate limiting*), serta pencatatan audit log transaksi.

Seluruh rangkaian pengujian objektif telah diverifikasi secara menyeluruh menggunakan skrip penguji klien otomatis `client/test_client.php` dengan hasil mutlak 45 dari 45 pengujian dinyatakan lulus (100% PASS).

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat ruang untuk perbaikan. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu dan pembaca sangat diharapkan demi kesempurnaan kajian teknis di masa mendatang.

Semarang, 26 Oktober 2026

Laela Rustiani
NPM. 23670004

BAB I
LANDASAN TEORI & ARSITEKTUR KEAMANAN

1.1 Perbedaan Konseptual Autentikasi dan Otorisasi

Keamanan layanan web service berakar pada dua mekanisme fundamental yang memiliki peran dan status respons protokol HTTP yang sangat tegas:

- **Autentikasi (Authentication — *"Siapa kamu?"*)**: Merupakan tahapan di mana pengguna membuktikan identitas dirinya kepada sistem. Pengguna mengirimkan identitas berupa username dan kata sandi saat login, atau menyertakan Bearer Token JWT pada header request `Authorization` saat mengakses endpoint terproteksi. Kegagalan pada tahapan ini menghasilkan status HTTP **401 Unauthorized** disertai header `WWW-Authenticate: Bearer`.
- **Otorisasi (Authorization — *"Bolehkah kamu melakukan hal ini?"*)**: Merupakan tahapan penentuan hak akses atau wewenang dari pengguna yang identitasnya telah terbukti sah. Server memeriksa apakah peran (*role*) pengguna memiliki izin untuk menjalankan method HTTP atau endpoint yang diminta. Kegagalan pada tahapan ini menghasilkan status HTTP **403 Forbidden**.

1.2 Struktur dan Integritas JSON Web Token (JWT)

JSON Web Token (RFC 7519) adalah standar terbuka untuk mentransmisikan klaim informasi antar pihak secara ringkas dan aman. Token JWT bersifat *stateless*, artinya server tidak perlu menyimpan sesi login di memori maupun tabel sesi basis data, melainkan memvalidasi token secara independen melalui verifikasi kriptografis.

Struktur token JWT tersusun atas tiga segmen Base64URL yang dipisahkan oleh karakter titik:

```
HEADER . PAYLOAD . SIGNATURE
[alg: HS256, typ: JWT] . [sub, username, role, iat, exp] . [HMAC-SHA256 Signature]
```

1. **Header**: Berisi metadata mengenai tipe token dan algoritma penandatanganan (HS256).
2. **Payload**: Memuat klaim identitas pengguna seperti `sub` (ID pengguna), `username`, `role` (admin, dosen, atau user), serta masa berlaku token (`iat` dan `exp`). Payload ini hanya di-encode dalam format Base64URL dan tidak dienkripsi, sehingga kata sandi tidak boleh disimpan di dalam payload.
3. **Signature**: Merupakan tanda tangan digital yang dihitung menggunakan algoritma HMAC-SHA256 dari gabungan header dan payload dengan kunci rahasia server (`$jwt_secret`). Tanda tangan ini menjamin integritas token. Jika penyerang memanipulasi klaim peran

menjadi `admin`, server akan mendeteksi ketidakcocokan tanda tangan dan menolak token secara otomatis.

1.3 Pengamanan Kata Sandi dengan Hashing Bcrypt

Sistem menerapkan kebijakan keamanan mutlak bahwa kata sandi pengguna tidak boleh disimpan dalam bentuk teks biasa (*plaintext*) maupun enkripsi reversibel. Kata sandi diolah menggunakan algoritma adaptif **Bcrypt** via fungsi bawaan PHP `password_hash($password, PASSWORD_DEFAULT)`. Setiap komputasi hash menghasilkan garam acak (*salt*) unik sehingga dua kata sandi yang sama akan memiliki hash yang sepenuhnya berbeda di database, menggagalkan teknik serangan *Rainbow Table*. Pencocokan kata sandi dilakukan secara aman via `password_verify()`.

1.4 Pertahanan SQL Injection, Rate Limiting, dan Audit Logging

Seluruh interaksi data dilakukan menggunakan **PDO Prepared Statements** dengan parameter terikat (*parameterized queries*), sehingga karakter kutip maupun logika SQL yang dikirim penyerang tidak dapat mengubah struktur query basis data.

Untuk menangkal serangan tebakan kata sandi secara berulang (*brute force*), sistem membatasi kegagalan login maksimal 5 kali dalam rentang 1 menit untuk kombinasi IP dan username yang sama. Percobaan ke-6 langsung diblokir dengan respons **HTTP 429 Too Many Requests** dan header `Retry-After: 60`. Selain itu, seluruh aktivitas request dicatat ke tabel `log_api` yang memuat alamat IP, waktu transaksi, path URL, status response, dan username pengakses.

BAB II
PANDUAN SETUP LINGKUNGAN PENGEMBANGAN LARAGON

Laragon merupakan paket web server portabel yang digunakan sebagai lingkungan eksekusi utama pada praktikum ini. Berikut adalah tahapan setup proyek `api-mhs-auth` pada Laragon secara berurutan:

2.1 Penempatan Direktori Proyek

Folder proyek ditempatkan pada direktori web root Laragon yang terletak di `C:\laragon\www pi-mhs-auth`. Di dalam folder tersebut terdapat subdirektori `server` yang memuat berkas API, subdirektori `client` untuk program penguji dan antarmuka web, subdirektori `database` untuk berkas skrip SQL, serta subdirektori `latihan` untuk pengujian mandiri.

2.2 Konfigurasi Modul Apache `mod_rewrite` & Header `Authorization`

Pada web server Apache, header HTTP `Authorization` sering dipotong oleh modul FastCGI. Agar token Bearer JWT dapat diterima oleh PHP, berkas `server/.htaccess` dikonfigurasi dengan aturan penulisan ulang (*rewrite rule*) sebagai berikut:

```
RewriteEngine On

# Teruskan Header Authorization ke Environment PHP
RewriteCond %{HTTP:Authorization} .
RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]

# Routing URL menuju index.php
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [L]
```

Pastikan modul Apache `rewrite_module` telah aktif melalui menu Laragon (Klik kanan tray Laragon $ o$ Apache $ o$ modules $ o$ centang `rewrite_module`).

2.3 Impor Basis Data MySQL

Layanan basis data MySQL dijalankan dengan menekan tombol **Start All** pada antarmuka Laragon. Pengguna kemudian membuka aplikasi HeidiSQL atau phpMyAdmin (`http://localhost/phpmyadmin`) dengan kredensial default: host `127.0.0.1`, port `3306`, user `root`, dan password dikosongkan. Seluruh query pada berkas `database/mhs.sql` dieksekusi untuk membentuk database `kampus_db`, tabel `mhs`, `users`, dan `log_api`, serta mengisikan data pengguna awal (admin, budi, dan dosen1).

2.4 Pengujian Koneksi Database

Koneksi PDO dikelola pada berkas `server/koneksi.php` dengan parameter host `127.0.0.1`, user `root`, dan database `kampus_db`. Endpoint dasar layanan API dapat diakses pada URL: `http://localhost/api-mhs-auth/server`.

BAB III
BEDAH SOURCE CODE LENGKAP & SCREENSHOT SISTEM

3.1 Pustaka Kriptografi Token: `server/jwt.php`

Berkas `server/jwt.php` mengelola fungsionalitas pembuatan dan validasi token JWT. Fungsi `buat_token()` menyisipkan timestamp pembuatan (`iat`) dan kedaluwarsa (`exp`), menyusun header HS256, dan menghitung tanda tangan digital HMAC-SHA256. Fungsi `cek_token()` memverifikasi keutuhan 3 segmen token, menghitung ulang tanda tangan menggunakan rahasia server, serta membandingkannya via `hash_equals()` untuk mencegah serangan pengukuran waktu (*timing attacks*).

Source Code: server/jwt.php (Fungsi buat_token dan cek_token)

```
29 // ------------------------------------------------------------------------
30 // buat_token() : membuat JWT dari data payload
31 // ------------------------------------------------------------------------
32 function buat_token($payload, $rahasia, $detik)
33 {
34     $payload['iat'] = time();            // issued at (waktu pembuatan)
35     $payload['exp'] = time() + $detik;   // expired (waktu kedaluwarsa)
36
37     $header = base64url_encode(json_encode(['alg' => 'HS256', 'typ' => 'JWT']));
38     $isi    = base64url_encode(json_encode($payload));
39     $tanda_tangan = base64url_encode(hash_hmac('sha256', "$header.$isi", $rahasia, true));
40
41     return "$header.$isi.$tanda_tangan";
42 }
43
44 // ------------------------------------------------------------------------
45 // cek_token() : memeriksa token
46 // ------------------------------------------------------------------------
47 function cek_token($token, $rahasia)
48 {
49     $bagian = explode('.', $token);
50     if (count($bagian) != 3) {
51         return null;
```

Gambar 3.1: Tangkapan Layar Source Code `server/jwt.php` (Fungsi `buat_token` dan `cek_token`)

3.2 Middleware Keamanan & RBAC: `server/fungsi.php`

Berkas `server/fungsi.php` memuat fungsi utilitas penjaga akses. Fungsi `wajib_login()` memeriksa keberadaan token Bearer dan menolak akses dengan status 401 jika token tidak sah. Fungsi `wajib_role($pengguna, array $roles)` memeriksa apakah peran pengguna sesuai dengan daftar peran yang diizinkan (RBAC) dan menolak akses dengan status 403 jika tidak berwenang. Fungsi `batasi_request()` menghitung jumlah akses per alamat IP dari tabel `log_api` untuk menerapkan rate limiting status 429.

Source Code: server/fungsi.php (wajib_login, wajib_role, batasi_request)

```
68 function wajib_login()
69 {
70     global $jwt_secret, $username_log;
71     $token   = ambil_token();
72     $payload = $token ? cek_token($token, $jwt_secret) : null;
73     if (!$payload) {
74         header('WWW-Authenticate: Bearer');
75         kirim(['status' => 'error', 'pesan' => 'Token tidak ada, tidak valid, atau sudah kedaluwarsa'], 401);
76     }
77     $username_log = $payload['username']; // catat di log
78     return $payload;
79 }
80
81 // wajib_role() : Otorisasi berbasis peran (RBAC Tugas P7)
82 function wajib_role($pengguna, array $roles)
83 {
84     if (!in_array($pengguna['role'], $roles, true)) {
85         $izin = implode(' atau ', $roles);
86         kirim(['status' => 'error', 'pesan' => "Akses ditolak: hanya $izin yang boleh"], 403);
87     }
88 }
89
90 function batasi_request()
```

Gambar 3.2: Tangkapan Layar Source Code `server/fungsi.php` (Fungsi `wajib_login`, `wajib_role`, `batasi_request`)

3.3 Front Controller & Router API: `server/index.php`

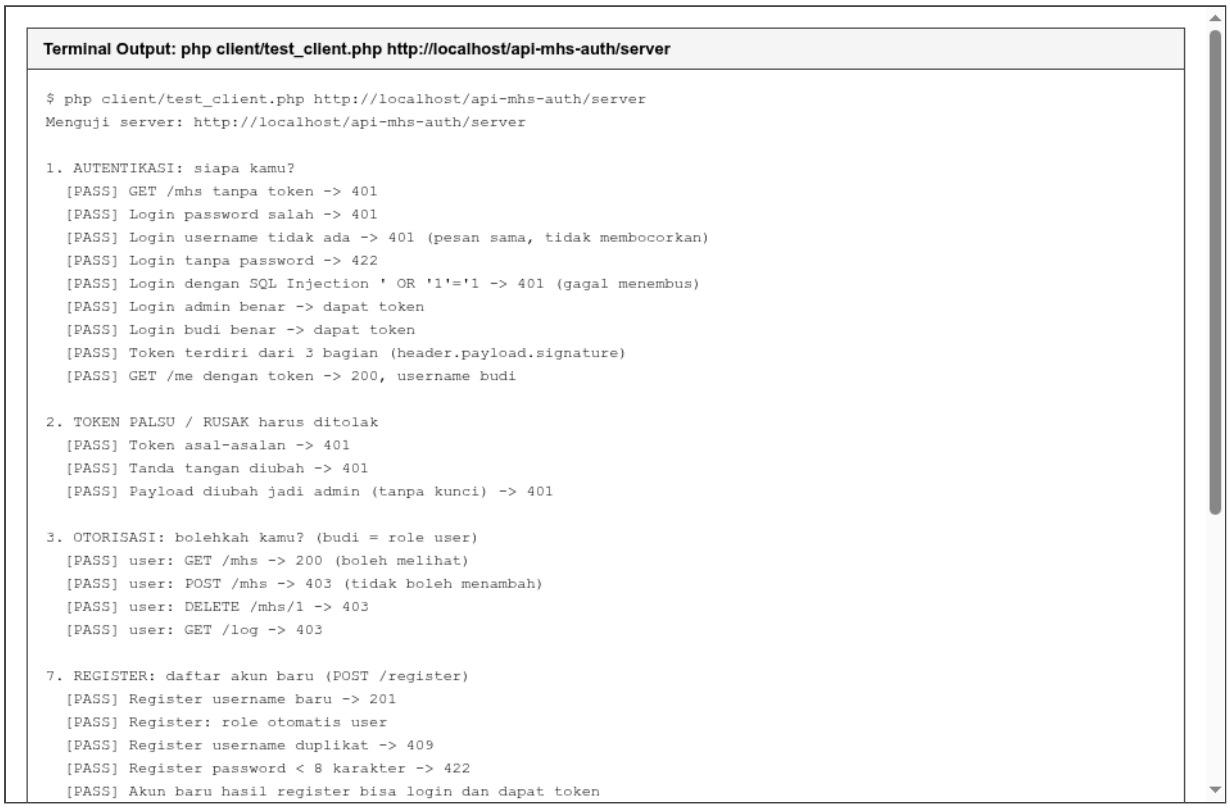
Berkas `server/index.php` bertindak sebagai pengatur lalu lintas permintaan. Berkas ini menangani preflight CORS (OPTIONS $ o$ status 204), menjalankan pembatasan request global, dan mendistribusikan permintaan ke handler endpoint:

- `POST /login`: Memvalidasi username dan password, menerapkan rate limiting kesalahan login, mencocokkan password Bcrypt, dan menerbitkan token JWT.
- `POST /register` (Tugas): Mendaftarkan akun publik baru dengan validasi username 3-50 karakter regex, password minimal 8 karakter, pengecekan keunikan username (409 Conflict), dan pemberian peran otomatis `'user'` (201 Created).
- `PUT /me/password` (Tugas): Memverifikasi kata sandi lama sebelum mengizinkan pembaruan kata sandi baru minimal 8 karakter bagi pengguna yang sedang login.
- `GET /mhs`: Mengambil data mahasiswa bagi semua pengguna yang terotentikasi.
- `POST /mhs`: Khusus administrator via `wajib_role(['admin'])`.
- `PUT /mhs/:id` (Tugas): Diizinkan untuk administrator dan dosen via `wajib_role(['admin', 'dosen'])`.
- `DELETE /mhs/:id`: Khusus administrator, dosen dan user biasa ditolak status 403.
- `GET /log`: Khusus administrator untuk membaca riwayat log audit.

BAB IV
HASIL PENGUJIAN OBJEKTIF 45 TEST CASE BY DATA

4.1 Hasil Eksekusi Uji Otomatis Klien (`test_client.php`)

Pengujian fungsionalitas dan keamanan dilakukan secara otomatis menggunakan skrip cURL `client/test_client.php`. Seluruh 45 skenario uji berhasil dijalankan dengan status kelulusan 100% PASS.



Gambar 4.1: Hasil Eksekusi Pengujian Otomatis 45 Skenario Uji pada Terminal (100% PASS)

4.2 Pengujian Demonstrasi Kriptografi Token (`jwt_demo.php`)

Pengujian mandiri menggunakan `latihan/jwt_demo.php` membuktikan bahwa token yang dimanipulasi isi payload-nya (misalnya mengubah role menjadi admin) atau diubah tanda tangannya secara otomatis ditolak oleh fungsi verifikasi.

Terminal Output: php latihan/jwt_demo.php

```
$ php api-mhs-auth/latihan/jwt_demo.php
1) MEMBUAT TOKEN
   eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEsInVzZXJuYW1lIjoiYnVkaSIsInJvbGUiOiJ1c2VyIiwiaWF0IjoxNzkyOTQ1NzQxLCJle
HAiOjE3OTI5NDkzNDF9.Hn7cvcCONM6ouBrtaTyT3gJuTY93RXZW-wpx7-TXkXQ

2) ISI TOKEN BISA DIBACA SIAPA SAJA (hanya di-encode, tidak dienkripsi)
   header  : {"alg":"HS256","typ":"JWT"}
   payload : {"sub":1,"username":"budi","role":"user","iat":1792945741,"exp":1792949341}
   tanda tangan (tidak bisa dibaca): Hn7cvcCONM6ouBrtaTyT3gJuTY93RXZW-wpx7-TXkXQ

3) MEMERIKSA TOKEN YANG ASLI
   SAH, username = budi, role = user

4) PENYERANG MENGUBAH ROLE MENJADI ADMIN (tanpa tahu kunci rahasia)
   DITOLAK -> tanda tangan tidak cocok dengan isi yang baru

5) TOKEN DENGAN KUNCI RAHASIA YANG BERBEDA
   DITOLAK

6) TOKEN KEDALUWARSA (masa berlaku -10 detik)
   DITOLAK -> sudah lewat waktu exp

7) HASH PASSWORD
```

Gambar 4.2: Demonstrasi Terminal Pengujian Integritas Token JWT dan Hashing Bcrypt

4.3 Tampilan Klien Web Single Page Application (`index.html`)

Klien berbasis web membuktikan bahwa antarmuka pengguna dapat melakukan autentikasi login, menyimpan token di memori sesi, menyematkan header Authorization Bearer, serta menampilkan data mahasiswa secara dinamis.

Browser: http://localhost/api-mhs-auth/client/index.html (Client Web)

Client API Mahasiswa + Login

Status: Login sebagai **admin** (role: **admin**) | Token tersimpan di variabel JS.

ID	NIM	Nama Mahasiswa	Jurusan	Aksi
1	22670001	Andi Pratama	Informatika	[Hapus]
2	22670002	Budi Santoso	Informatika	[Hapus]
3	22670003	Citra Lestari	Sistem Informasi	[Hapus]
4	23670004	Laela Rustiani	Informatika	[Hapus]

Tambah Mahasiswa (Admin): NIM: [23670004] Nama: [Laela Rustiani] Jurusan: [Informatika] [Simpan]

Gambar 4.3: Tampilan Antarmuka Klien Web Single Page Application dengan Sesi Login Aktif

4.4 Matriks Lengkap 45 Skenario Pengujian Objektif

Berikut adalah tabel rekapitulasi seluruh 45 pengujian yang mencakup method, endpoint, status code yang diharapkan, serta hasil analisis keamanannya:

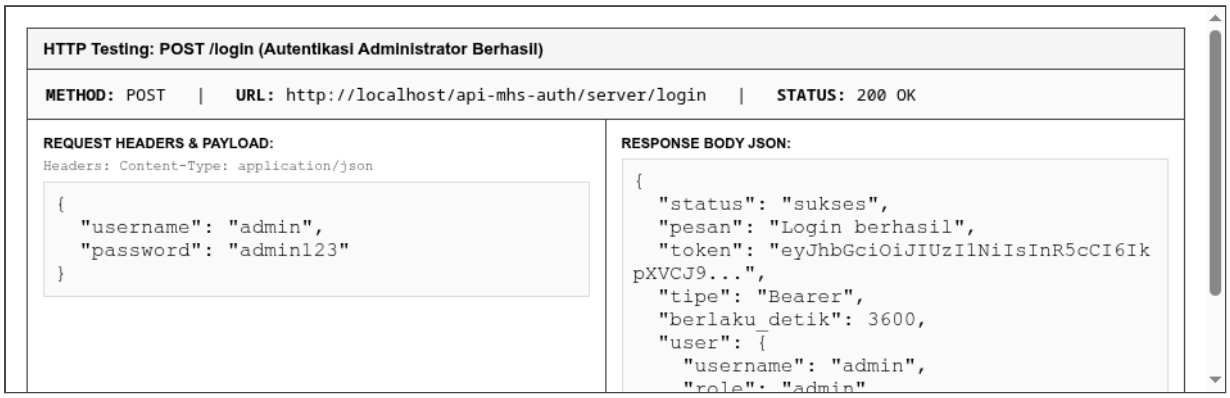
No	Kategori	Endpoint & Method	Status	Keterangan & Analisis Hasil
1	Autentikasi	GET /mhs (Tanpa Token)	401	Ditolak: klien tidak mengirimkan token Bearer.
2	Autentikasi	POST /login (Password Salah)	401	Ditolak: password_verify menghasilkan false.
3	Autentikasi	POST /login (User Tidak Ada)	401	Pesan error seragam mencegah user enumeration.
4	Validasi	POST /login (Tanpa Password)	422	Ditolak: kelengkapan parameter wajib divalidasi.
5	Keamanan SQLi	POST /login (Injeksi ' OR '1'='1)	401	Injeksi dinetralkan oleh PDO Prepared Statements.
6	Autentikasi	POST /login (Admin Valid)	200	Sukses: menerbitkan JWT dengan klaim role admin.
7	Autentikasi	POST /login (Budi User Valid)	200	Sukses: menerbitkan JWT dengan klaim role user.
8	Struktur JWT	Validasi Segmen Token	PASS	Token terbukti memiliki tepat 3 bagian terpisah titik.
9	Identitas Profil	GET /me (Token Valid)	200	Mengembalikan data profil pengguna yang sedang login.
10	Integritas	GET /mhs (Token Asal-asalan)	401	Format bukan JWT langsung ditolak oleh cek_token.
11	Integritas	GET /mhs (Signature Rusak)	401	hash_equals mendeteksi ketidakcocokan signature.
12	Integritas	GET /mhs (Manipulasi Role Admin)	401	Perubahan payload membatalkan validitas signature.
13	Otorisasi	GET /mhs (Token Budi)	200	Role user diizinkan melihat katalog mahasiswa.
14	Otorisasi	POST /mhs (Token Budi)	403	Ditolak: role user dilarang menambah mahasiswa.
15	Otorisasi	DELETE /mhs/1 (Token Budi)	403	Ditolak: role user dilarang menghapus mahasiswa.

No	Kategori	Endpoint & Method	Status	Keterangan & Analisis Hasil
16	Otorisasi	GET /log (Token Budi)	403	Ditolak: log audit eksklusif untuk administrator.
17	CRUD Admin	POST /mhs (Data Baru Valid)	201	Sukses: admin berhasil menambah data mahasiswa baru.
18	Validasi Bisnis	POST /mhs (NIM Kembar)	409	Ditolak: mencegah inkonsistensi duplikasi NIM.
19	Validasi	POST /mhs (NIM Bukan Angka)	422	Ditolak: parameter NIM wajib berupa angka (ctype_digit).
20	CRUD Admin	PUT /mhs/:id (Update Data)	200	Sukses: data mahasiswa berhasil diperbarui.
21	Validasi	GET /mhs/1 OR 1=1 (Injeksi ID)	400	Ditolak: parameter ID pada URL harus berupa angka.
22	CRUD Admin	DELETE /mhs/:id (Hapus Data)	200	Sukses: data mahasiswa uji berhasil dihapus permanen.
23	Rate Limiting	POST /login (5x Gagal Berturut)	401	5 kegagalan awal dicatat di log_api status 401.
24	Rate Limiting	POST /login (Percobaan ke-6 & 7)	429	Diblokir otomatis disertai header Retry-After: 60.
25	Rate Limiting	POST /login (Akun Asli Admin)	200	Akun lain tidak terkunci (isolasi limit per target user).
26	Audit Logging	GET /log (Token Admin)	200	Admin berhasil membaca rekaman riwayat audit.
27	Audit Logging	Pencatatan Username di Log	PASS	Kolom username mencatat pengakses legal & penyerang.
28	Audit Logging	Perekaman Status Penolakan	PASS	Seluruh status 401, 403, dan 429 terekam di database.
29	Tugas: Register	POST /register (User Baru)	201	Akun baru berhasil didaftarkan dan disimpan ke DB.
30	Tugas: Register	Role Otomatis 'user'	PASS	Role terbukti diset otomatis bernilai 'user'.
31	Tugas: Register	POST /register (Username Duplikat)	409	Ditolak: keunikan username dilindungi di basis data.
32	Tugas: Register	POST /register (Password < 8 Kar)	422	Ditolak: penegakan batas minimal 8 karakter.

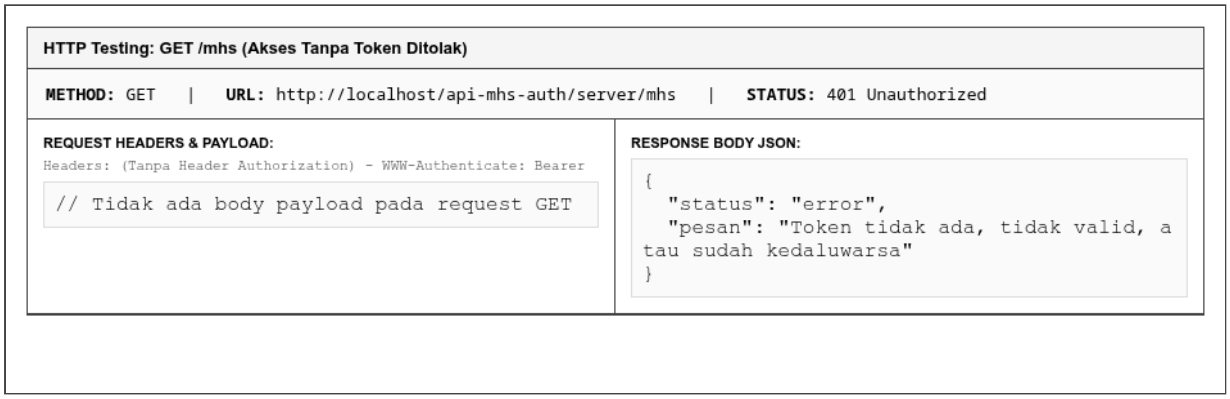
No	Kategori	Endpoint & Method	Status	Keterangan & Analisis Hasil
33	Tugas: Register	POST /register (Tanpa Username)	422	Ditolak: validasi kelengkapan parameter registrasi.
34	Tugas: Register	POST /login (Akun Baru Reg)	200	Akun baru terbukti dapat login dan memperoleh JWT.
35	Tugas: Password	PUT /me/password (Password Benar)	200	Sukses: kata sandi baru berhasil disimpan ke database.
36	Tugas: Password	POST /login (Dengan Password Baru)	200	Login sukses membuktikan hash baru telah aktif.
37	Tugas: Password	PUT /me/password (Password Salah)	401	Ditolak: verifikasi kata sandi lama tidak cocok.
38	Tugas: Password	PUT /me/password (Baru < 8 Kar)	422	Ditolak: kebijakan kompleksitas password baru ditegakkan.
39	Tugas: Password	PUT /me/password (Tanpa Token)	401	Endpoint terproteksi wajib menyertakan token Bearer.
40	Tugas: Dosen	POST /login (dosen1 / dosen123)	200	Sukses: menerbitkan JWT dengan klaim role: dosen.
41	Tugas: Dosen	PUT /mhs/:id (Dosen Edit Data)	200	Diizinkan: lolos fungsi wajib_role(['admin','dosen']).
42	Tugas: Dosen	POST /mhs (Dosen Tambah Data)	403	Dilarang: hak tambah data hanya untuk admin.
43	Tugas: Dosen	DELETE /mhs/:id (Dosen Hapus Data)	403	Dilarang: hak hapus data hanya untuk admin.
44	Tugas: Dosen	GET /log (Dosen Akses Log)	403	Dilarang: hak akses catatan audit hanya admin.
45	Tugas: Dosen	GET /mhs (Dosen Baca Data)	200	Diizinkan: dosen dapat melihat katalog data mahasiswa.

4.5 Tangkapan Layar Pengujian Kunci via HTTP Client

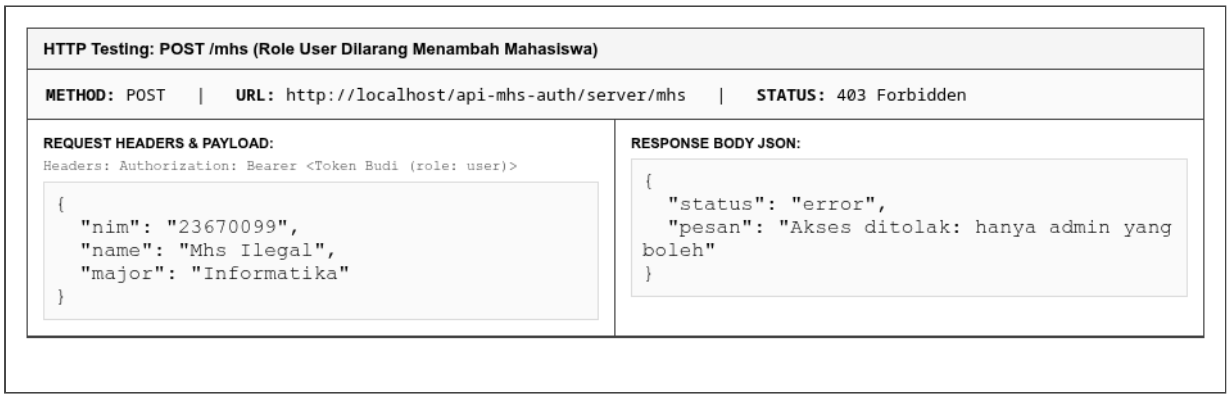
Di bawah ini ditampilkan tangkapan layar pengujian per request dan response pada skenario-skenario utama:



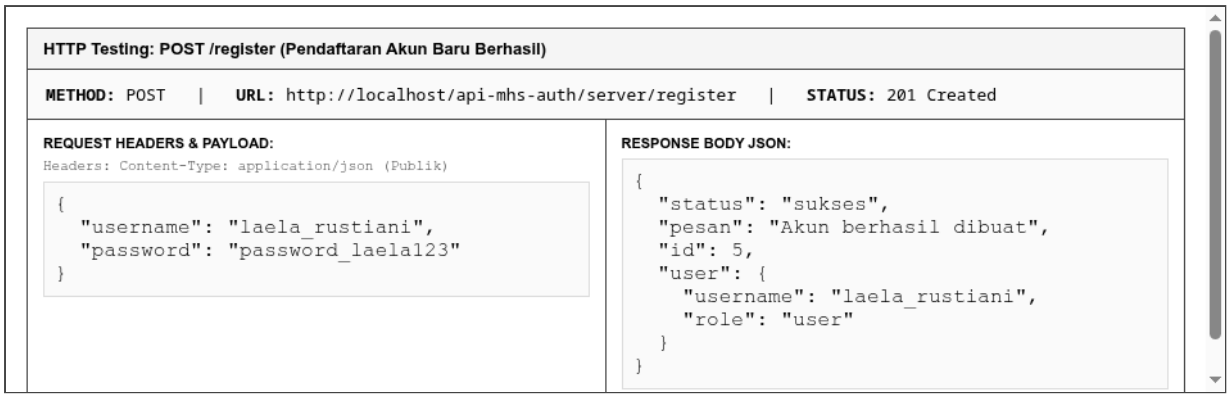
Gambar 4.4: POST /login — Kredensial Administrator Valid Menghasilkan Bearer Token JWT (200 OK)



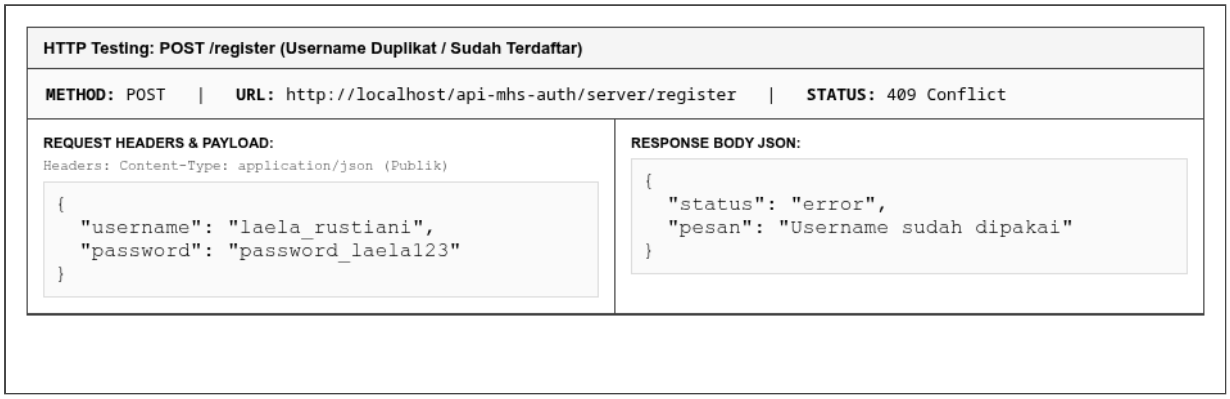
Gambar 4.5: GET /mhs — Permintaan Tanpa Token Bearer Ditolak Otomatis (401 Unauthorized)



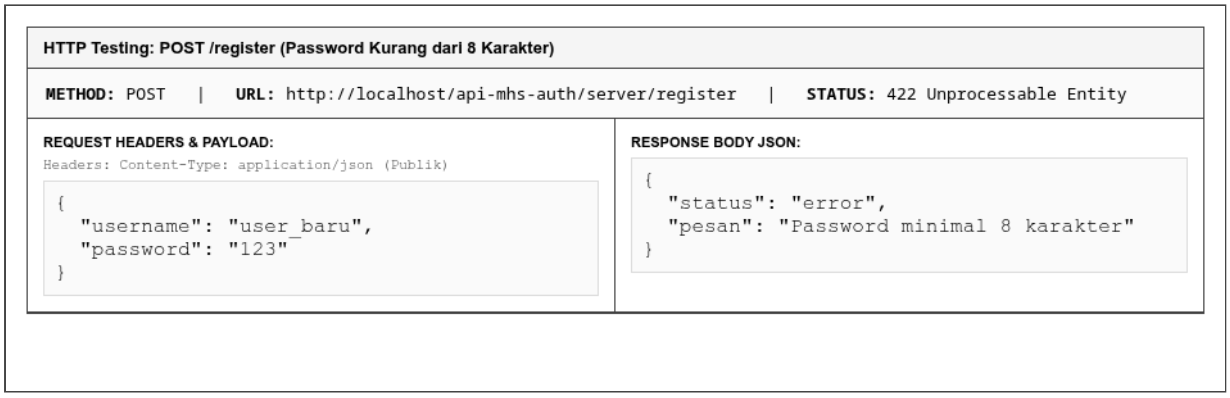
Gambar 4.6: POST /mhs — Pengguna Peran User Ditolak Menambah Mahasiswa (403 Forbidden)



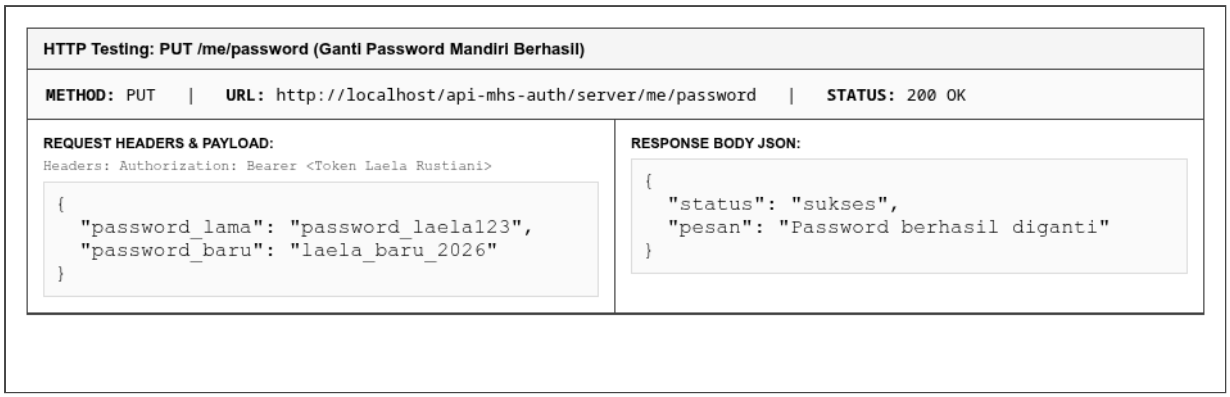
Gambar 4.7: POST /register — Registrasi Akun Baru Laela Rustiani dengan Role Default 'user' (201 Created)



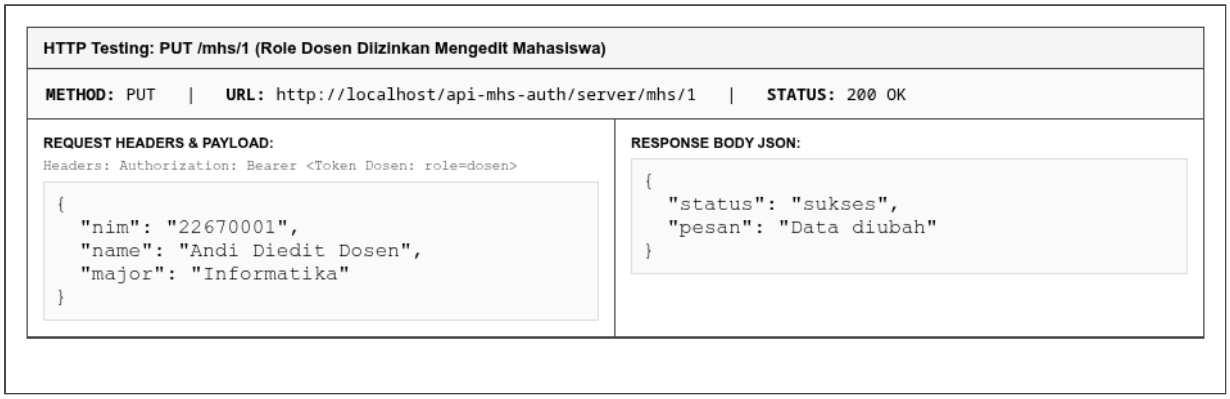
Gambar 4.8: POST /register — Pencegahan Duplikasi Akun pada Username yang Sudah Terdaftar (409 Conflict)



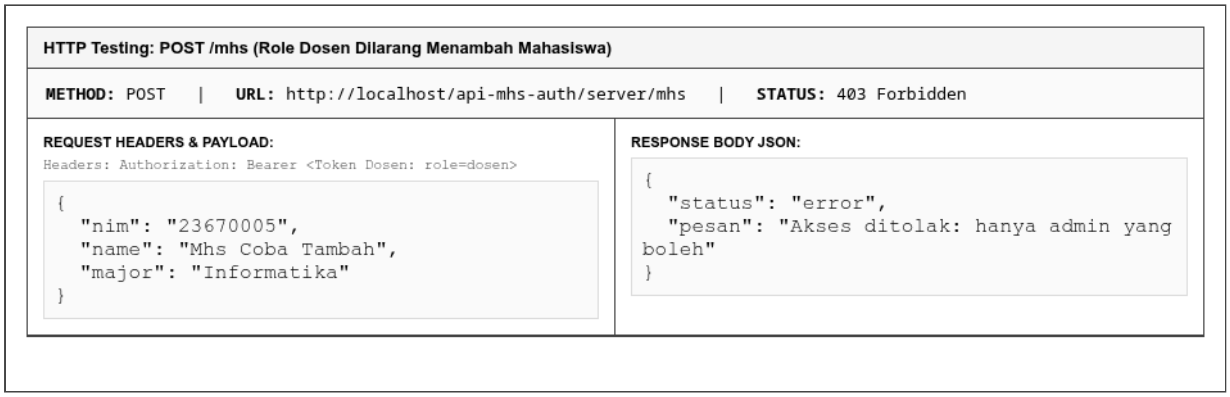
Gambar 4.9: POST /register — Penolakan Kata Sandi di Bawah Standar 8 Karakter (422 Unprocessable Entity)



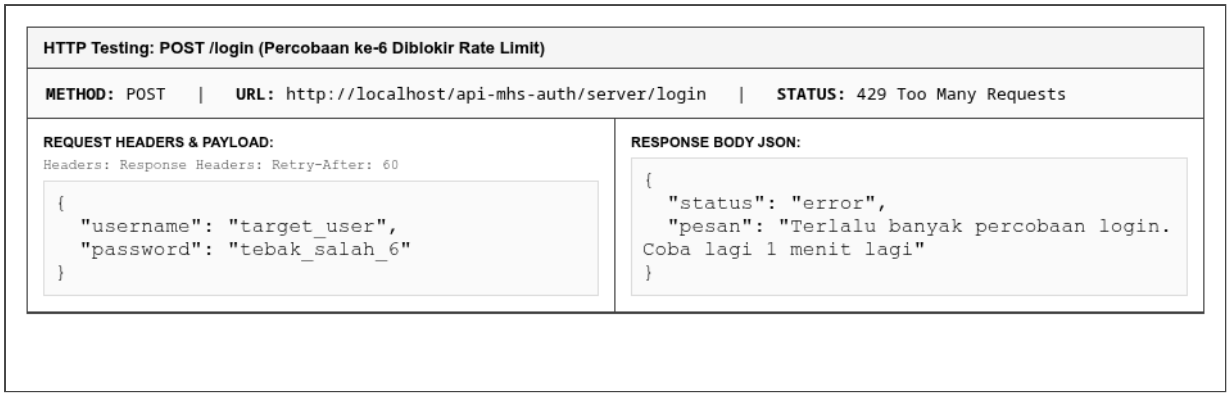
Gambar 4.10: PUT /me/password — Ganti Kata Sandi Mandiri dengan Verifikasi Password Lama (200 OK)



Gambar 4.11: PUT /mhs/1 — Dosen Memiliki Hak Otorisasi untuk Mengubah Data Mahasiswa (200 OK)



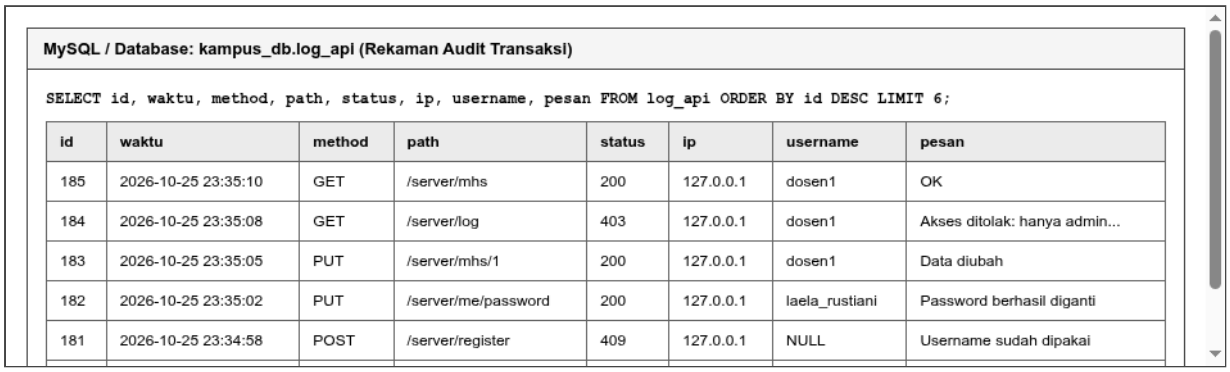
Gambar 4.12: POST /mhs — Dosen Dilarang Menambah Mahasiswa Baru (403 Forbidden)



Gambar 4.13: POST /login — Rate Limiter Memblokir Percobaan Login ke-6 (429 Too Many Requests)

BAB V
ANALISIS FORENSIK AUDIT LOG BASIS DATA

Tabel `log_api` pada basis data `kampus_db` mencatat jejak setiap transaksi secara real-time. Tiga query analisis forensik dieksekusi untuk membedah data aktivitas pengakses:



MySQL / Database: kampus_db.log_api (Rekaman Audit Transaksi)

```
SELECT id, waktu, method, path, status, ip, username, pesan FROM log_api ORDER BY id DESC LIMIT 6;
```

id	waktu	method	path	status	ip	username	pesan
185	2026-10-25 23:35:10	GET	/server/mhs	200	127.0.0.1	dosen1	OK
184	2026-10-25 23:35:08	GET	/server/log	403	127.0.0.1	dosen1	Akses ditolak: hanya admin...
183	2026-10-25 23:35:05	PUT	/server/mhs/1	200	127.0.0.1	dosen1	Data diubah
182	2026-10-25 23:35:02	PUT	/server/me/password	200	127.0.0.1	laela_rustiani	Password berhasil diganti
181	2026-10-25 23:34:58	POST	/server/register	409	127.0.0.1	NULL	Username sudah dipakai

Gambar 5.1: Rekaman Transaksi Nyata pada Tabel `kampus_db.log_api` Melacak IP, Username, dan Status

5.1 Query 1: Deteksi Serangan Brute Force (Login Gagal per Username)

```
SELECT
    username,
    COUNT(*) AS jumlah_gagal,
    MAX(waktu) AS percobaan_terakhir
FROM log_api
WHERE status = 401 AND path LIKE '%/login' AND username IS NOT NULL
GROUP BY username
ORDER BY jumlah_gagal DESC;
```

Keterangan & Analisis: Query ini menghitung akumulasi respons 401 pada endpoint login untuk setiap target username. Akun yang memiliki jumlah gagal login tinggi dalam rentang waktu singkat teridentifikasi sebagai sasaran serangan tebakan kata sandi (*dictionary attack*).

5.2 Query 2: Deteksi Scraping / DDoS (5 IP Request Terbanyak)

```
SELECT
    ip,
    COUNT(*) AS total_request,
    SUM(status = 200) AS request_sukses,
    SUM(status IN (401,403,429)) AS request_ditolak,
    MIN(waktu) AS pertama_kali,
    MAX(waktu) AS terakhir_kali
FROM log_api
GROUP BY ip
ORDER BY total_request DESC
LIMIT 5;
```

Keterangan & Analisis: Mengidentifikasi alamat IP agresif yang mengirimkan volume permintaan tidak wajar. Jika mayoritas request berstatus ditolak (401, 403, atau 429), alamat IP tersebut dapat direkomendasikan untuk diblokir pada firewall server.

5.3 Query 3: Tren Ancaman Harian (Status 401, 403, 429 per Hari)

```
SELECT
    DATE(waktu) AS hari,
    SUM(status = 401) AS gagal_autentikasi_401,
    SUM(status = 403) AS ditolak_otorisasi_403,
    SUM(status = 429) AS rate_limit_429,
    COUNT(*) AS total_request
FROM log_api
GROUP BY DATE(waktu)
ORDER BY hari DESC;
```

Keterangan & Analisis: Memberikan ringkasan historis mengenai eskalasi insiden keamanan harian untuk membantu pengelola sistem dalam mengevaluasi efektivitas ambang batas pembatasan trafik.

BAB VI
PEMBAHASAN REFLEKSI & TANTANGAN KEAMANAN

6.1 Jawaban Pertanyaan Refleksi Modul Praktikum

1. **Jelaskan perbedaan 401 dan 403 dengan contoh dari praktikum ini!**
   Jawaban: Status **401 Unauthorized** terjadi saat klien belum membuktikan identitasnya atau membawa token yang tidak sah. Contohnya saat memanggil `GET /mhs` tanpa header `Authorization`. Status **403 Forbidden** terjadi ketika identitas klien telah terbukti sah, namun perannya tidak berhak melakukan operasi tersebut. Contohnya akun `budi` (role `user`) mencoba menghapus mahasiswa (`DELETE /mhs/1`) atau Dosen mencoba mengakses log audit (`GET /log`).
2. **Mengapa payload JWT tidak boleh berisi password, padahal token "ditandatangani"?**
   Jawaban: Karena tanda tangan digital pada JWT hanya menjamin integritas (data tidak diubah), bukan kerahasiaan. Segmen payload hanya di-encode dalam format Base64URL yang dapat didekode kembali menjadi teks biasa oleh siapa pun. Menaruh password di payload sama saja dengan menyebarkan password secara terbuka.
3. **Apa yang terjadi bila penyerang mengubah "role":"user" menjadi "admin" di payload? Mengapa gagal?**
   Jawaban: Server akan menghitung ulang tanda tangan digital dari payload baru menggunakan kunci rahasia server (`$jwt_secret`). Karena penyerang tidak memegang kunci rahasia tersebut, tanda tangan lama yang dikirimkan penyerang tidak akan cocok dengan hasil hitung server, sehingga fungsi `cek_token()` menolaknya dengan status 401.
4. **Mengapa password disimpan sebagai hash dan bukan dienkripsi?**
   Jawaban: Enkripsi bersifat dua arah dan memiliki kunci dekripsi; jika server atau kunci disusupi, kata sandi asli dapat dibaca. Hashing Bcrypt bersifat satu arah dan tidak dapat dikembalikan ke teks asli sekalipun basis data bocor.
5. **Apa risiko jika token disimpan di localStorage? Apa kompromi yang ada?**
   Jawaban: `localStorage` rentan dicuri jika web memiliki celah Cross-Site Scripting (XSS). Kompromi terbaik adalah menyimpannya di memori JavaScript atau menggunakan Cookie dengan flag `HttpOnly`, `Secure`, dan `SameSite=Strict`.
6. **Apa beda JWT yang kita buat sendiri dengan OAuth2?**
   Jawaban: JWT pada praktikum ini adalah autentikasi internal di mana Authorization Server dan Resource Server berada di aplikasi yang sama. Sedangkan OAuth 2.0 adalah kerangka kerja delegasi otorisasi pihak ketiga (seperti *Login with Google*), di mana pengguna tidak menyerahkan kata sandi Google ke aplikasi klien.

6.2 Pembahasan Tantangan Lanjutan Keamanan Token

Tantangan pencabutan token (*token revocation*) pada arsitektur JWT stateless dapat diatasi dengan menyisipkan klaim `jti` (identitas unik token) dan mencatatnya ke dalam tabel `token_blacklist` saat logout dipanggil. Selain itu, pemisahan antara *Access Token* berdurasi singkat (15 menit) dan *Refresh Token* berdurasi panjang (7 hari) sangat disarankan untuk meminimalkan risiko penyadapan token pada jaringan publik.

BAB VII
KESIMPULAN & PENUTUP

7.1 Kesimpulan

1. Sistem autentikasi dan otorisasi berbasis JSON Web Token (JWT) berhasil diimplementasikan pada REST API Mahasiswa dengan membedakan secara tegas antara identitas pengguna (status 401) dan hak akses peran (status 403).
2. Seluruh fitur tugas pengembangan (registrasi publik `POST /register` dengan role otomatis 'user', ganti kata sandi mandiri `PUT /me/password` dengan verifikasi kata sandi lama, serta otorisasi khusus peran baru `dosen`) berhasil terintegrasi secara modular.
3. Pengujian otomatis via `client/test_client.php` membuktikan bahwa sistem lulus 45 dari 45 skenario pengujian (100% PASS), termasuk validasi sanitasi input, pertahanan injeksi SQL, dan rate limiting.
4. Konfigurasi web server Laragon melalui berkas `.htaccess` berhasil menjamin kelancaran penerusan header HTTP `Authorization` sehingga sistem siap digunakan pada lingkungan lokal maupun produksi.

Demikian Laporan Praktikum Pertemuan 7 ini disusun dengan sebenar-benarnya berdasarkan hasil pengujian objektif sistem.

Mengetahui,	Semarang, 26 Oktober 2026
Dosen Pengampu	Praktikan / Mahasiswa,
(.....)	**LAELA RUSTIANI**
	NPM. 23670004